

✓ Practical No 06

Name : Pranjal somatkar	roll no. : B 48
-------------------------	-----------------

Aim: To extract data from a website using Python libraries such as Requests and BeautifulSoup.

✓ Theory

1. What is Web Scraping? Explain the role of the Requests library in web scraping.

****Web Scraping – ****

Web scraping is a technique used to automatically extract large amounts of data from websites. Instead of manually copying information, a program (usually written in Python) accesses web pages, reads their content, and collects useful data in a structured format such as CSV files, Excel sheets, or databases.

Web scraping is widely used in many real-world applications such as:

- **E-commerce:** Price comparison between different websites
- **Research:** Collecting data for analysis
- **News Aggregation:** Gathering headlines from multiple sources
- **Social Media Analysis:** Extracting trends and user data
- **Job Portals:** Collecting job listings automatically

How Web Scraping Works (Step-by-Step)

1. A request is sent to a website
 2. The website responds with HTML content
 3. The HTML is parsed and processed
 4. Required data is extracted
 5. Data is stored in a structured format
-

Role of the Requests Library in Web Scraping

The **Requests library** is one of the most important tools in Python web scraping. It is used to send HTTP requests to websites and retrieve their content.

Without Requests, your program cannot communicate with websites.

Functions of Requests Library

- Sends HTTP requests like GET and POST
- Retrieves webpage content (HTML, JSON, etc.)
- Handles headers and cookies
- Manages authentication and sessions
- Works with APIs

Basic Example Using Requests

```
import requests

url = "https://example.com"

response = requests.get(url)

print("Status Code:", response.status_code)
print("Web Page Content:\n", response.text)
```

Explanation of the Code

- `requests.get(url)` → Sends request to website
- `response.status_code` → Shows if request is successful (200 = OK)
- `response.text` → Contains full HTML content

Common Methods in Requests

Method	Description
<code>get()</code>	Fetch data from a website
<code>post()</code>	Send data to server
<code>put()</code>	Update data
<code>delete()</code>	Delete data
<code>status_code</code>	Check request status
<code>text</code>	Get HTML content
<code>json()</code>	Convert response into JSON

Example with Headers (Advanced Use)

Some websites block bots. Requests allows adding headers to act like a browser.

```
import requests
```

```
url = "https://example.com"

headers = {
    "User-Agent": "Mozilla/5.0"
}

response = requests.get(url, headers=headers)

print(response.text)
```

Why Requests is Important in Web Scraping

Feature	Benefit
Easy to use	Simple syntax
Fast	Quick data retrieval
Flexible	Supports APIs and websites
Powerful	Handles sessions and cookies

Limitations of Requests

- Cannot parse HTML (needs BeautifulSoup)
- Cannot handle JavaScript-heavy websites
- Only fetches raw data

2. What is BeautifulSoup? How does it help in parsing HTML?

Introduction to BeautifulSoup

BeautifulSoup is a Python library used for extracting and parsing data from HTML and XML documents. It is especially useful in web scraping because it allows developers to easily navigate, search, and modify the structure of web pages.

When a webpage is downloaded using a tool like the Requests library, it returns raw HTML code, which is often complex and difficult to understand. BeautifulSoup simplifies this by converting the HTML into a structured format called a **parse tree**, making it easier to extract specific information.

Why BeautifulSoup is Needed

HTML documents contain nested tags, attributes, and content. Manually extracting data from such structures is difficult. BeautifulSoup solves this problem by:

- Organizing HTML into a tree structure
- Allowing easy navigation between elements

- Providing simple methods to search and extract data

How BeautifulSoup Helps in Parsing HTML

Parsing means analyzing and breaking down HTML into smaller parts so that useful data can be extracted.

Steps Involved:

1. Fetch webpage using Requests
2. Pass HTML content to BeautifulSoup
3. Convert HTML into parse tree
4. Search for required elements
5. Extract text or attributes

Example HTML

```
<html>
  <head><title>My Page</title></head>
  <body>
    <h1>Welcome</h1>
    <p class="info">This is a paragraph.</p>
  </body>
</html>
```

Example Code Using BeautifulSoup

```
from bs4 import BeautifulSoup
import requests

url = "https://example.com"
response = requests.get(url)

soup = BeautifulSoup(response.text, "html.parser")

# Extract title
print(soup.title.text)

# Extract heading
print(soup.find("h1").text)

# Extract paragraph
print(soup.find("p", class_="info").text)
```

Explanation of Code

- `BeautifulSoup(response.text, "html.parser")` → Parses HTML
- `soup.title.text` → Extracts title
- `find()` → Finds first matching tag
- `class_="info"` → Searches by class

Common Parsing Methods in BeautifulSoup

Method	Description
<code>find()</code>	Returns first matching element
<code>find_all()</code>	Returns all matching elements
<code>select()</code>	Uses CSS selectors
<code>get_text()</code>	Extracts text only
<code>attrs</code>	Access attributes like href, class

Example: Extracting Links

```
links = soup.find_all("a")

for link in links:
    print(link.get("href"))
```

Features of BeautifulSoup

- Easy to learn and use
- Works with poorly formatted HTML
- Supports different parsers (`html.parser`, `lxml`)
- Provides powerful search functions

Advantages and Limitations

Advantages

- Simplifies HTML parsing
- Flexible and powerful
- Beginner-friendly

Limitations

- Slower than some advanced tools like Scrapy
- Cannot fetch data itself (needs Requests)
- Not suitable for JavaScript-rendered pages

3. Differentiate Between Web Scraping and Web Crawling

Introduction

Web scraping and web crawling are two closely related concepts in data extraction from the internet, but they serve different purposes. While both involve accessing web pages, their goals, methods, and outcomes are quite different.

What is Web Scraping?

Web scraping refers to the process of extracting specific data from websites. It focuses on collecting useful information such as text, images, prices, or links from selected web pages.

Example: Extracting product prices from an e-commerce website or collecting news headlines.

What is Web Crawling?

Web crawling refers to the process of systematically browsing the internet to discover and index web pages. It is mainly used by search engines to explore websites and build a database of web content.

Example: Search engines scanning websites to index pages for search results.

Key Differences Between Web Scraping and Web Crawling

Feature	Web Scraping	Web Crawling
Definition	Extracting specific data from websites	Discovering and indexing web pages
Purpose	Data collection	Website exploration
Scope	Narrow and focused	Broad and extensive
Data Handling	Structured data extraction	URL collection and indexing
Usage	Research, price comparison, analytics	Search engines like Google
Tools	BeautifulSoup, Requests, Scrapy	Crawlers, bots, spiders
Speed	Depends on data extraction	Continuous and large-scale

Working Difference

Web Scraping Process

1. Identify target website
2. Send request to server
3. Extract required data
4. Store structured data

Web Crawling Process

1. Start with a list of URLs
 2. Visit each page
 3. Extract links from pages
 4. Continue visiting new links
 5. Index content
-

Example to Understand Easily

- **Web Scraping** is like going to a specific shop and collecting only the items you need.
 - **Web Crawling** is like exploring every shop in a city to make a directory of all shops.
-

Real-World Applications

Web Scraping

- Price monitoring tools
- Data analysis
- Content aggregation

Web Crawling

- Search engine indexing
 - Website auditing
 - SEO analysis
-

Relationship Between Scraping and Crawling

Web crawling and web scraping often work together:

- Crawlers first discover web pages
- Scrapers then extract useful data from those pages

4. What are the commonly used methods in BeautifulSoup?

Introduction

BeautifulSoup provides a variety of methods that make it easy to navigate, search, and extract data from HTML and XML documents. These methods allow users to locate specific tags, extract text, and access attributes efficiently, which is essential in web scraping.

1. `find()` Method

The `find()` method is used to locate the **first occurrence** of a tag that matches given criteria.

Syntax

```
soup.find(tag_name, attributes)
```

Example

```
soup.find("h1")
```

Explanation

- Returns only the first matching element
 - Useful when only one result is needed
-

2. `find_all()` Method

The `find_all()` method returns **all matching elements** in the document.

Syntax

```
soup.find_all(tag_name)
```

Example

```
soup.find_all("p")
```

Explanation

- Returns a list of elements
 - Useful for extracting multiple data items
-

3. `select()` Method

The `select()` method allows searching using **CSS selectors**, similar to web development.

Syntax

```
soup.select("selector")
```

Example


```
soup.select(".class_name")  
soup.select("#id_name")
```

Explanation

- Powerful and flexible
- Can target complex HTML structures

4. `get_text()` Method

This method extracts only the **text content** from HTML tags, removing all tags.

Example

```
soup.get_text()
```

Explanation

- Useful for cleaning data
- Removes HTML formatting

5. `get()` Method

Used to retrieve the value of an attribute.

Example

```
link.get("href")
```

Explanation

- Extracts attributes like links, images, etc.

6. `attrs` Property

Provides access to all attributes of a tag as a dictionary.

Example

```
tag.attrs
```

7. Navigation Methods

These methods help move through the HTML structure.

Method	Description
<code>parent</code>	Access parent element

Method	Description
<code>children</code>	Access child elements
<code>next_sibling</code>	Next element
<code>previous_sibling</code>	Previous element

8. `prettify()` Method

Formats HTML into a readable structure.

Example

```
print(soup.prettify())
```

Summary Table of Methods

Method	Purpose
<code>find()</code>	First matching element
<code>find_all()</code>	All matching elements
<code>select()</code>	CSS selector search
<code>get_text()</code>	Extract text
<code>get()</code>	Get attribute value
<code>attrs</code>	Access attributes
<code>parent/children</code>	Navigation
<code>prettify()</code>	Format HTML

Example Program Using Multiple Methods

```
from bs4 import BeautifulSoup
import requests

url = "https://example.com"
response = requests.get(url)

soup = BeautifulSoup(response.text, "html.parser")

# Extract title
print(soup.title.get_text())

# Extract all links
links = soup.find_all("a")

for link in links:
    print(link.get("href"))
```

5. What are the ethical and legal considerations in web scraping?

Introduction

Web scraping is a powerful technique for extracting data from websites, but it must be used responsibly. Ethical and legal considerations are important to ensure that scraping does not harm websites, violate laws, or misuse data. Ignoring these considerations can lead to penalties, blocked access, or even legal action.

A. Ethical Considerations in Web Scraping

Ethics refers to doing what is morally right while collecting data.

1. Respect Website Terms of Service

- Every website has rules (Terms of Service or ToS)
 - Scraping against these rules is unethical
 - Always read and follow the website policies
-

2. Follow robots.txt

- Websites provide a `robots.txt` file to guide bots
- It tells which pages can or cannot be accessed

Example:

```
User-agent: *  
Disallow: /private/
```

Meaning: Do not access the `/private/` section

3. Avoid Overloading the Server

- Sending too many requests can crash a website
- Use delays between requests

Example:

```
import time  
time.sleep(2)
```

4. Do Not Scrape Sensitive Data

- Avoid collecting:

- Personal data
- Financial details
- Confidential information

5. Give Proper Credit

- If data is used publicly, mention the source
- Avoid plagiarism

B. Legal Considerations in Web Scraping

Legal considerations involve following laws and regulations.

1. Copyright Laws

- Website content (text, images, videos) may be protected
- Copying without permission can lead to legal issues

2. Data Privacy Laws

- Laws like GDPR (Europe) and IT laws in India protect user data
- Personal data cannot be collected without consent

3. Terms of Service Violations

- Violating ToS can result in:
 - Account bans
 - IP blocking
 - Legal notices

4. Unauthorized Access

- Scraping login-protected or restricted pages is illegal
- Avoid bypassing security mechanisms

C. Ethical vs Legal Comparison

Aspect	Ethical Consideration	Legal Consideration
Focus	Moral responsibility	Law and regulations
Example	Not overloading servers	Following copyright laws
Violation Result	Reputation damage	Legal penalties

D. Best Practices for Safe Web Scraping

1. Always check website policies
 2. Use polite scraping (slow requests)
 3. Identify your bot using headers
 4. Avoid restricted or private data
 5. Use APIs if available instead of scraping
 6. Store and use data responsibly
-

Example with Headers and Delay

```
import requests
import time

url = "https://example.com"

headers = {
    "User-Agent": "Mozilla/5.0"
}

response = requests.get(url, headers=headers)
print(response.status_code)

time.sleep(2)
```

Experiment: Web Scrapping using Python (BeautifulSoup)

Step-by-Step Procedure

Step 1: Install Required Libraries

```
!pip install requests beautifulsoup4
```

```
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages  
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-packages  
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages  
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages  
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages  
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages  
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages  
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages
```

Step 2: Import Libraries

```
import requests
from bs4 import BeautifulSoup
import csv
```

Step 3: Define the Website URL

```
url = "http://books.toscrape.com/catalogue/page-1.html"
```

Step 4: Send HTTP Request

```
response = requests.get(url)

print("Status Code:", response.status_code)
```

```
Status Code: 200
```

Step 5: Parse HTML Content

```
soup = BeautifulSoup(response.text, "html.parser")
```

Step 6: Extract Required Data (Book Names)

```
books = soup.find_all("h3")

book_names = []

for b in books:
    book_names.append(b.find("a")["title"])
```

Step 7: Create Author Data (For Dataset)

```
authors = [
    "Author 1", "Author 2", "Author 3", "Author 4", "Author 5",
    "Author 6", "Author 7", "Author 8", "Author 9", "Author 10"
]
```

Step 8: Display Extracted Data

```
for i in range(10):
    print("Book:", book_names[i])
    print("Author:", authors[i])
    print()
```

```
Book: A Light in the Attic
Author: Author 1
```

Book: Tipping the Velvet
Author: Author 2

Book: Soumission
Author: Author 3

Book: Sharp Objects
Author: Author 4

Book: Sapiens: A Brief History of Humankind
Author: Author 5

Book: The Requiem Red
Author: Author 6

Book: The Dirty Little Secrets of Getting Your Dream Job
Author: Author 7

Book: The Coming Woman: A Novel Based on the Life of the Infamous Feminist, V
Author: Author 8

Book: The Boys in the Boat: Nine Americans and Their Epic Quest for Gold at t
Author: Author 9

Book: The Black Maria
Author: Author 10

Step 9: Store Data into CSV File (Dataset Creation)

```
with open("books_dataset.csv", "w", newline="", encoding="utf-8") as file:  
    writer = csv.writer(file)  
    writer.writerow(["Book Name", "Author"])  
  
    for i in range(10):  
        writer.writerow([book_names[i], authors[i]])  
  
print("Dataset saved successfully!")
```

Dataset saved successfully!

✓ Flowchart

```
graph TD  
    Start([Start]) --> Install[Install Libraries]  
    Install --> Import[Import Modules]  
    Import --> Send[Send Request to Website]
```



```
↓  
Receive HTML Content  
↓  
Parse HTML using BeautifulSoup  
↓  
Extract Required Data  
↓  
Store Data in CSV File  
↓  
End
```

Conclusion

The web scraping experiment using Python was successfully performed using the **Requests** and **BeautifulSoup** libraries. The program was able to connect to a website, retrieve its HTML content, and parse it to extract useful information such as book titles and related data.

Through this experiment, it was understood that web scraping does not require a predefined dataset, as the website itself acts as a data source. The extracted data was further organized and stored in a CSV file, thereby creating a structured dataset for future use.

This practical also helped in understanding key concepts such as sending HTTP requests, parsing HTML documents, and extracting specific elements using methods like `find()` and `find_all()`. Additionally, it highlighted the importance of ethical practices while performing web scraping.

Overall, the experiment demonstrates that Python provides an efficient and simple way to collect and manage web-based data, making it highly useful for data analysis, research, and real-world applications.